Food Store — NoSQL y Nuevas Formas de Almacenamiento

Base de Datos II · Unidad 7 · Tecnicatura Universitaria en Programación (UTN)

Estudiante: Cristian Rodriguez (Cloverfield)
Repositorio: github.com/Cristian0w0/BaseDeDatos2 (Rama: tp9-nosql)
Motor RDBMS: PostgreSQL 17.11 (Base: foodstore)
Motor Documental: MongoDB 9.0.4 (mongosh 2.x, Base: foodstore_nosql)

PARTE 1 · MODELO DOCUMENTAL Y MIGRACIÓN

Consigna 1: Selección del Subconjunto y Diseño del Modelo Documental

Para representar la gestión comercial de **Food Store** se seleccionó el subconjunto transaccional central: **pedidos** y sus **detalles de pedido**, enriquecido con la información relevante de **clientes** y **productos** vigente al momento de la venta.

1. Identificación del subconjunto relacional en PostgreSQL (schema.sql)

Tabla	Clave Primaria (PK)	Claves Foráneas (FK)	Descripción, Atributos Clave y Restricciones
pedido	id_pedido (IDENTITY)	cliente_id → cliente(id_cliente) [RESTRICT]	Cabecera: fecha_pedido (TIMESTAMPTZ), forma_pago (ENUM), estado (ENUM: PENDIENTE, CONFIRMADO, ENTREGADO, CANCELADO).
detalle_pedido	(pedido_id, producto_id)	pedido_id → pedido [RESTRICT] producto_id → producto [RESTRICT]	Líneas de venta: cantidad (>0) y precio_unitario (≥0, precio histórico pactado al comprar).
cliente	id_cliente (IDENTITY)	Ninguna	Entidad compradora: nombre, apellido, email (UNIQUE, length > 0).
producto	id_producto (IDENTITY)	categoria_id → categoria [RESTRICT]	Catálogo: nombre, precio (precio actual de lista), stock, activo.

> **Regla de integridad relacional:** En PostgreSQL no se almacenan subtotales ni totales (atributos derivados). La cardinalidad mínima (todo pedido debe poseer al menos un detalle) se garantiza mediante el disparador diferido trg_validar_pedido_con_detalle.

2. Esquema propuesto para la colección documental en MongoDB (Colección: pedidos)

Se adopta un diseño desnormalizado basado en el patrón **Aggregate Root**, consolidando cada orden en un único documento BSON autocontenido:

- **Identificador de origen conservado (_id):** Se preserva el id_pedido original de PostgreSQL mediante NumberLong(id_pedido), garantizando correspondencia biunívoca entre ambos motores.
- **Campos raíz:** fecha_pedido (ISODate), forma_pago (string enum), estado ("ENTREGADO", "CONFIRMADO", "PENDIENTE") para ciclo de vida y filtros de entrega, y total (numérico desnormalizado para reportería directa).
- **Objeto embebido (cliente):** Subdocumento con cliente_id, nombre, apellido y email. Captura la foto descriptiva del cliente evitando búsquedas cruzadas ($lookup).
- **Arreglo de líneas (lineas):** Arreglo con producto_id, nombre_producto, cantidad, precio_unitario histórico y subtotal.

```
{
  "_id": NumberLong("1"),
  "fecha_pedido": ISODate("2026-10-10T12:30:00Z"),
  "estado": "ENTREGADO",
  "forma_pago": "EFECTIVO",
  "total": 100.00,
  "cliente": {
    "cliente_id": NumberLong("1"),
    "nombre": "Cliente",
    "apellido": "Inicial",
    "email": "cliente.inicial@foodstore.local"
  },
  "lineas": [
    { "producto_id": NumberLong("1"), "nombre_producto": "Producto Inicial", "cantidad": 1, "precio_unitario": 100.00, "subtotal": 100.00 }
  ]
}
```

Consigna 2: Creación de Base, Colección y Migración Documental

Se estructuró y ejecutó el script `consultas_mongodb.js` en la base de datos `foodstore_nosql` sobre la colección `pedidos`. Se migraron 5 pedidos representativos con sus líneas de detalle completas y datos esenciales de clientes.

Fragmento sintético de migración ejecutado con insertMany():

```
db = db.getSiblingDB("foodstore_nosql");
db.pedidos.drop(); // Idempotencia de ejecución

db.pedidos.insertMany([
  {
    "_id": NumberLong("1"), "fecha_pedido": ISODate("2026-10-15T12:00:00Z"), "estado": "ENTREGADO", "forma_pago": "EFECTIVO", "total":
100.00,
    "cliente": { "cliente_id": NumberLong("1"), "nombre": "Cliente", "apellido": "Inicial", "email": "cliente.inicial@foodstore.local" },
    "lineas": [
      { "producto_id": NumberLong("1"), "nombre_producto": "Producto Inicial", "cantidad": 1, "precio_unitario": 100.00, "subtotal": 100.00
}
    ]
  },
  {
    "_id": NumberLong("2"), "fecha_pedido": ISODate("2026-10-16T15:30:00Z"), "estado": "CONFIRMADO", "forma_pago": "TARJETA", "total":
200.00,
    "cliente": { "cliente_id": NumberLong("1"), "nombre": "Cliente", "apellido": "Inicial", "email": "cliente.inicial@foodstore.local" },
    "lineas": [
      { "producto_id": NumberLong("1"), "nombre_producto": "Producto Inicial", "cantidad": 2, "precio_unitario": 100.00, "subtotal": 200.00
}
    ]
  },
  // Pedidos #3 ($175.50), #4 ($268.00) y #5 ($377.50) cargados con estructura idéntica (ver consultas_mongodb.js)
]);
```

Evidencia de ejecución en MongoDB Shell:

```
$ mongosh --quiet --file docs/tp9/consultas_mongodb.js
Total de documentos en la colección pedidos: 5 ✓ Éxito
```

Consigna 3: Reglas de Integridad y Justificación de Datos Embebidos

1. Reglas de integridad conservadas durante la migración

1. **Inmutabilidad del precio histórico pactado:** En PostgreSQL, `detalle_pedido.precio_unitario` almacena el valor nominal al comprar (`CHECK precio_unitario ≥ 0`). En MongoDB, este valor y el subtotal quedan congelados de forma permanente en cada subdocumento de `lineas`. Futuras variaciones de precio o bajas lógicas en el catálogo no afectan los registros históricos.
2. **Cardinalidad mínima del pedido y cantidades positivas:** En PostgreSQL, el trigger diferido `trg_validar_pedido_con_detalle` y la restricción `CHECK cantidad > 0` impiden pedidos vacíos. En MongoDB se asegura la regla equivalente garantizando que el arreglo `lineas` contenga al menos un subdocumento (`lineas.length ≥ 1`) con cantidades enteras positivas.

2. Justificación técnica de la inclusión de datos embebidos vs. colecciones separadas

- **Líneas de detalle embebidas:** Existe una relación de *composición estricta*: los ítems carecen de ciclo de vida fuera del pedido. Al embeberlos en un arreglo, se aprovecha la **atomicidad a nivel de documento único** (single-document atomicity) de MongoDB, recuperando o persistiendo la orden en una sola operación física I/O sin necesidad de transacciones complejas ni uniones (`$lookup`).
- **Snapshot descriptivo de cliente embebido:** Embeber `nombre`, `apellido` y `email` dentro del subdocumento `cliente` captura la "foto" exacta de facturación y despacho. Permite resolver las consultas y pantallas operativas de pedidos inmediatamente sin peticiones adicionales a la colección de clientes.

PARTE 2 · CONSULTAS Y VERIFICACIÓN

Consigna 4: Ejecución de Consultas en MongoDB (mongosh)

Consulta 4a: Mostrar todos los pedidos, incluyendo su identificador y estado

Instrucción: `db.pedidos.find({}, { _id: 1, estado: 1, total: 1 });`

```
[
  { _id: Long('1'), estado: 'ENTREGADO', total: 100, cliente: { nombre: 'Cliente', apellido: 'Inicial' } },
  { _id: Long('2'), estado: 'CONFIRMADO', total: 200, cliente: { nombre: 'Cliente', apellido: 'Inicial' } },
  { _id: Long('3'), estado: 'PENDIENTE', total: 175.5, cliente: { nombre: 'María', apellido: 'Rodríguez' } },
  { _id: Long('4'), estado: 'ENTREGADO', total: 268, cliente: { nombre: 'Carlos', apellido: 'Pérez' } },
  { _id: Long('5'), estado: 'CONFIRMADO', total: 377.5, cliente: { nombre: 'Ana', apellido: 'González' } }
]
```

Consulta 4b: Filtrar pedidos por un estado que exista en los datos migrados (estado: 'ENTREGADO')

Instrucción: `db.pedidos.find({ estado: "ENTREGADO" }, { _id: 1, estado: 1, total: 1 });`

```
[
  { _id: Long('1'), estado: 'ENTREGADO', total: 100 },
  { _id: Long('4'), estado: 'ENTREGADO', total: 268 }
]
```

Resultado obtenido: **2 pedidos** recuperados (IDs: 1 y 4).

Consulta 4c: Buscar pedidos que contengan un producto determinado (producto_id: 1 - Producto Inicial)

Instrucción: `db.pedidos.find({ "lineas.producto_id": NumberLong("1") }, { _id: 1, estado: 1, "lineas.nombre_producto": 1 });`

```
[
  { _id: Long('1'), estado: 'ENTREGADO', lineas: [ { producto_id: Long('1'), nombre_producto: 'Producto Inicial' } ] },
  { _id: Long('2'), estado: 'CONFIRMADO', lineas: [ { producto_id: Long('1'), nombre_producto: 'Producto Inicial' } ] }
]
```

Resultado obtenido: **2 pedidos** recuperados (IDs: 1 y 2). Se filtró con notación de punto directamente sobre el arreglo embebido.

Consulta 4d: Buscar pedidos cuyo total sea superior a un importe elegido y justificar el umbral

Umbral elegido: $200,00 (`total > 200.00`).

> **Justificación técnica del umbral:** En el catálogo de bebidas y productos de Food Store, los pedidos individuales iniciales (pedidos 1 a 3) presentan subtotales de entre $100.00 y $200.00 (1 a 3 unidades vendidas). Establecer el umbral en $200.00 segmenta de forma precisa las órdenes de mayor volumen (pedidos 4 y 5 con 4 y 5 unidades respectivamente, alcanzando $268.00 y $377.50), permitiendo identificar y recuperar el 40% superior del lote migrado correspondiente a compras de consumo extendido o abastecimiento.

Instrucción: `db.pedidos.find({ total: { $gt: 200.00 } }, { _id: 1, total: 1, "cliente.nombre": 1, "cliente.apellido": 1 });`

```
[
  { _id: Long('4'), total: 268, cliente: { nombre: 'Carlos', apellido: 'Pérez' } },
  { _id: Long('5'), total: 377.5, cliente: { nombre: 'Ana', apellido: 'González' } }
]
```

Resultado obtenido: **2 pedidos** recuperados (Pedido #4 por $268.00 y Pedido #5 por $377.50).

Consigna 5: Consultas Equivalentes en PostgreSQL y Verificación Cruzada

Se formularon y ejecutaron en PostgreSQL (script `consultas_postgresql.sql`) las consultas análogas a los apartados **b)** y **c)** para cotejar la consistencia entre ambos motores sobre el mismo lote de 5 pedidos.

1. Sentencias SQL ejecutadas en PostgreSQL

```
-- Equivalente a 4b: Filtrar pedidos por estado 'ENTREGADO'
SELECT p.id_pedido, p.estado, p.fecha_pedido, p.forma_pago
FROM pedido p
WHERE p.estado = 'ENTREGADO' AND p.id_pedido BETWEEN 1 AND 5
ORDER BY p.id_pedido;

-- Equivalente a 4c: Buscar pedidos con producto_id = 1 (Producto Inicial)
SELECT DISTINCT p.id_pedido, p.estado, dp.producto_id
FROM pedido p
JOIN detalle_pedido dp ON p.id_pedido = dp.pedido_id
WHERE dp.producto_id = 1 AND p.id_pedido BETWEEN 1 AND 5
ORDER BY p.id_pedido;
```

2. Tabla comparativa de verificación (PostgreSQL vs. MongoDB)

Consulta Evaluada	Cant. en PostgreSQL	IDs en PostgreSQL	Cant. en MongoDB	IDs en MongoDB	¿Coinciden IDs?
Estado seleccionado (`'ENTREGADO'`)	**2**	`[1, 4]`	**2**	`[1, 4]`	✓ Sí
Producto seleccionado (`producto_id: 1`)	**2**	`[1, 2]`	**2**	`[1, 2]`	✓ Sí

3. Análisis técnico de diferencias detectadas

1. **Navegación de arreglos vs. Uniones relacionales (`JOIN + DISTINCT`):** En MongoDB, la consulta por producto inspecciona el arreglo interno mediante `"lineas.producto_id": 1` sin alterar la cardinalidad de los documentos retornados. En PostgreSQL, cruzar `pedido` con `detalle_pedido` genera una unión relacional cartesiana; si un pedido contuviera el producto en más de una línea o se consultaran múltiples condiciones, es mandatorio utilizar `DISTINCT` o `WHERE EXISTS` para no duplicar filas de cabecera.
2. **Atributos derivados vs. Precalculados:** En MongoDB el importe `total` y los subtotales se persisten en el documento (denormalización). En PostgreSQL, en virtud de la 3FN, el subtotal se computa al vuelo (`cantidad * precio_unitario`) y el total requiere agregaciones (`SUM`), priorizando la no-redundancia sobre el costo computacional de lectura.

Verificación de modelo: No se detectaron discrepancias numéricas ni de cardinalidad entre ambos motores; los identificadores recuperados en ambos entornos resultaron 100% coincidentes.

PARTE 3 · EVALUACIÓN DE LA TECNOLOGÍA E IA

Consigna 6: Consulta a Herramienta de IA sobre PostgreSQL vs. MongoDB

Se planteó a un asistente de arquitectura de IA una consulta técnica contemplando el modelo actual, el patrón de acceso y la coherencia histórica:

> `"Actúa como arquitecto de bases de datos senior. En nuestro sistema Food Store tenemos modelados los pedidos en PostgreSQL mediante 4 tablas normalizadas: pedido, detalle_pedido, cliente y producto. El patrón de acceso principal es la lectura frecuente de pedidos completos (cabecera con todas sus líneas) para visualización operativa y despacho, y existe una estricta necesidad de conservar la coherencia histórica de precios unitarios y cantidades frente a futuras modificaciones del catálogo general de precios.`
> `¿Conviene migrar este subconjunto transaccional a una colección de documentos en MongoDB (embebiendo cliente y líneas) o conservarlo en PostgreSQL? Compara ambos motores respecto al rendimiento de lectura, integridad de datos y costos de mantenimiento."`

Resumen de la respuesta técnica de la IA:

- *Lecturas:* Señaló que MongoDB es "categóricamente más rápido" al eliminar los `JOIN`s y recuperar pedidos completos en una sola lectura.
- *Integridad:* Afirmó que MongoDB no provee garantías ACID ni integridad referencial, advirtiendo sobre posibles inconsistencias y pérdida de precios históricos.
- *Recomendación:* Migrar a MongoDB si prima la velocidad de lectura, o quedarse en PostgreSQL si se prioriza la consistencia financiera.

Consigna 7: Contraste de Afirmaciones y Conclusión Técnica

1. Contraste de dos afirmaciones concretas de la IA

- **Afirmación 1 de la IA:** "*En MongoDB no existen garantías transaccionales y corrés el riesgo de perder la coherencia de cantidades y precios históricos.*"
  Contraste técnico: Falsa / Imprecisa. La documentación oficial de MongoDB (*Data Modeling - Embedding*) destaca la **atomicidad a nivel de documento único** (single-document atomicity). Al embeber las líneas y los precios dentro del documento del pedido, la escritura de cabecera y detalles es 100% atómica sin requerir transacciones distribuidas. El precio histórico queda congelado por diseño dentro del documento, desacoplado de mutaciones en el catálogo. Además, MongoDB 4.0+ soporta formalmente transacciones ACID multi-documento.
- **Afirmación 2 de la IA:** "*MongoDB es categóricamente superior en rendimiento a PostgreSQL para consultar pedidos porque elimina por completo los JOINs.*"
  Contraste técnico: Parcialmente engañosa. En PostgreSQL, un `JOIN` entre `pedido` y `detalle_pedido` con índices B-Tree en la PK y en `detalle_pedido(pedido_id)` se ejecuta en submilisegundos en la memoria compartida (*shared_buffers*). Si bien MongoDB evita el costo de ensamblar el join en consulta, la denormalización produce documentos más pesados que incrementan la transferencia de red y el consumo de RAM de la base, requiriendo además validar stock y estado en la aplicación.

2. Conclusión técnica final (177 palabras — Límite: hasta 200 palabras)

> Para el ecosistema de Food Store, la estrategia óptima no consiste en reemplazar PostgreSQL por MongoDB, sino en adoptar una **persistencia políglota** o mantener PostgreSQL si el volumen es moderado.
>
> 1. **Integridad transaccional y concurrencia:** La creación de pedidos requiere descontar stock de forma concurrente, validar categorías activas y garantizar pagos bajo aislamiento ACID estricto. PostgreSQL resuelve esto nativamente mediante restricciones `CHECK`, claves foráneas y disparadores diferidos, previniendo sobreventas o inconsistencias contables sin depender exclusivamente de validaciones en la capa de software.
>
> 2. **Patrón de acceso y escalabilidad de lectura:** Por otro lado, la consulta recurrente de pedidos consolidados para despacho, facturación o historial de clientes se beneficia de la atomicidad a nivel de documento de MongoDB. Al embeber líneas y precios históricos congelados, se eliminan uniones relacionales repetitivas y se agiliza la lectura horizontal sin impacto en el motor OLTP principal.
>
> En conclusión, resulta conveniente **combinar ambas tecnologías**: conservar PostgreSQL como núcleo transaccional de escritura y catálogo maestro, y sincronizar asincrónicamente los pedidos finalizados hacia MongoDB para alimentar paneles de consulta y visualización de alto tráfico.